



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Impact on programming

3.1 Summary and General Aspects

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs

1. Introduction
2. The protocol stack
3. Impact on programming of on-line games (4 sessions)
 - **Summary and general aspects**
 - Latency, jitter, and reliability
 - Latency hiding techniques
 - Scalability
4. Other stories

Remember from last class



CITM

- Packet sniffers or packet analyzers allow us to capture traffic coming from or sent to the network
- We learnt how to use Wireshark, which analyzes protocols from the link layer to upper layers
- Wireshark can be a tool to inspect and troubleshoot the games that we program
 - Encourage you to use it during the lab sessions to understand what is really happening



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

The fundamentals of on-line game programming

What may we need to send?

- On-line games basically need to transmit and receive data related (but not limited to) changes of state, whatever that means depending on the type of game, purpose, context:
 - Movements or attacks within the game
 - Control messages to start or finish games
 - Keep-alive updates
 - Sanity checks
 - Update of leaderboards or other game services
 - Micro-transactions
 - Stream of the actual video (e.g. Google Stadia)

What may we need to send?

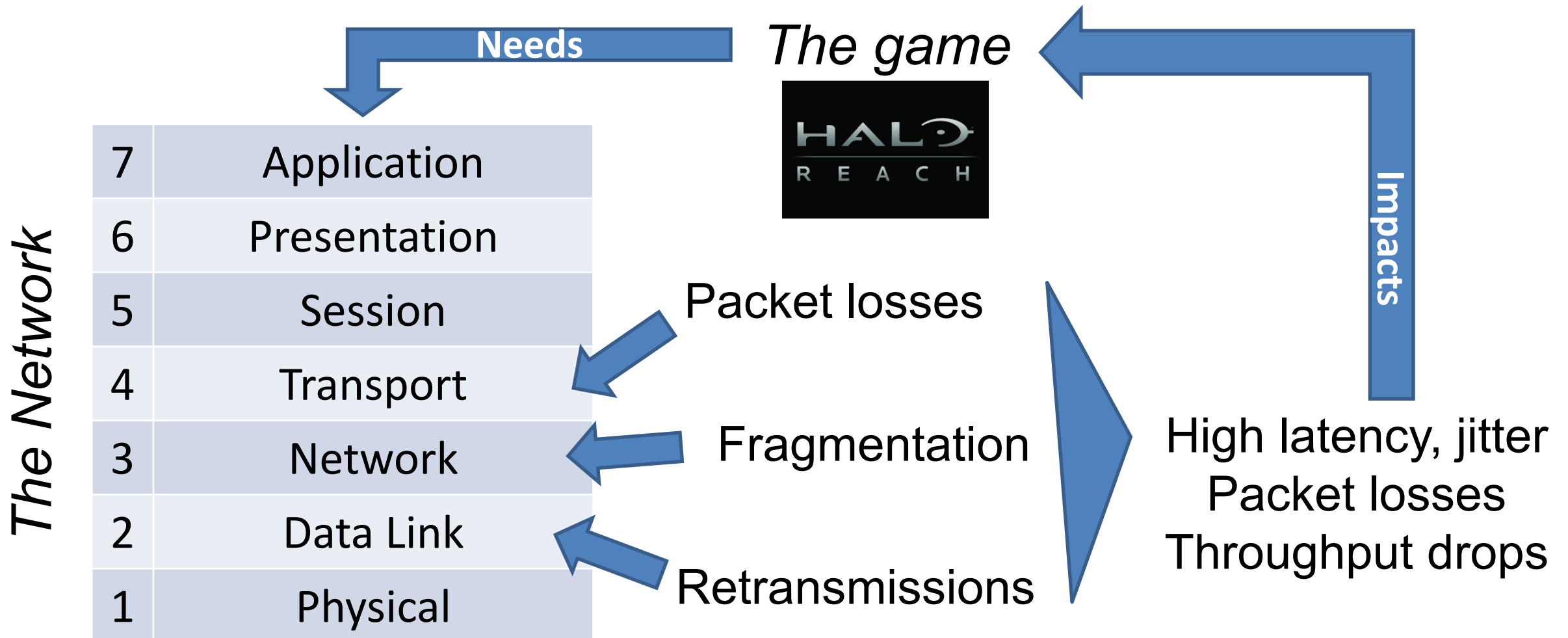
- Requirements depend on the type of message
 - Micro-transactions need to be reliable and secure (encrypted) as they involve payment data.
 - Messages for game services generally need to be reliable, and they are not latency sensitive.
 - Movements in games have different requirements depending on the type of game
 - **Turn-based:** little latency sensitivity (seconds)
 - **Real-Time Strategy (RTS):** some latency sensitivity (up to 1 second)
 - **Massive Multiplayer Games:** some latency sensitivity, also scalability
 - **Shooters or sport games:** a lot of latency sensitivity (up to 150 ms)
 - **AR/VR games:** super latency sensitive (<100 ms) to avoid cyber-sickness

Game against network



CITM

TCP is reliable but does not guarantee low latency



How do we program the game's netcode so that we can hide the detrimental effects of the network and still maintain as many functionalities as possible?

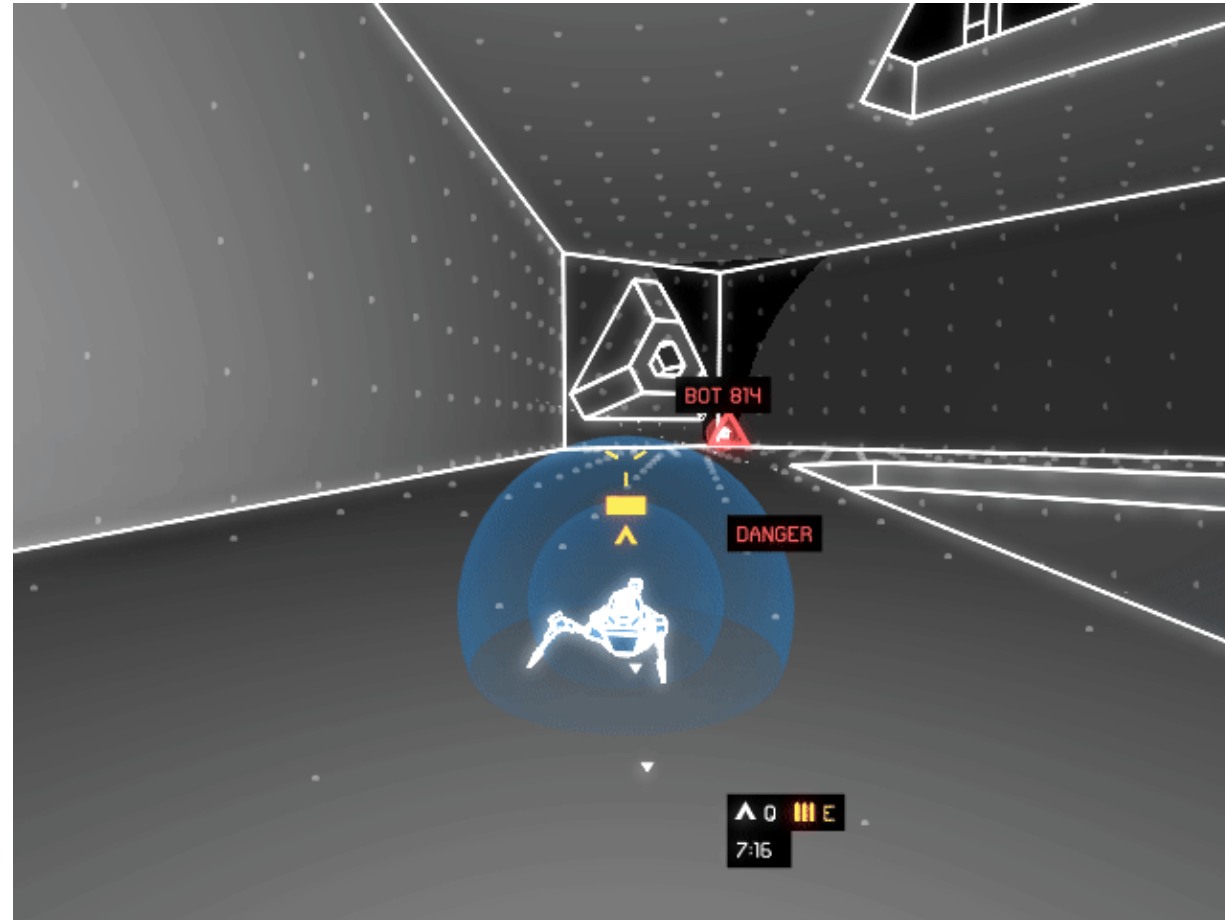
7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- On-line games need netcode (code in the game managing network-related issues) to exist
- The main goal is to maintain a consistent game state across all players in the most efficient way possible
- This process depends on:
 - The underlying topology or architecture
 - The type of game and the type of activity on it
 - The choices of the programmer on what is important

State, world, objects...



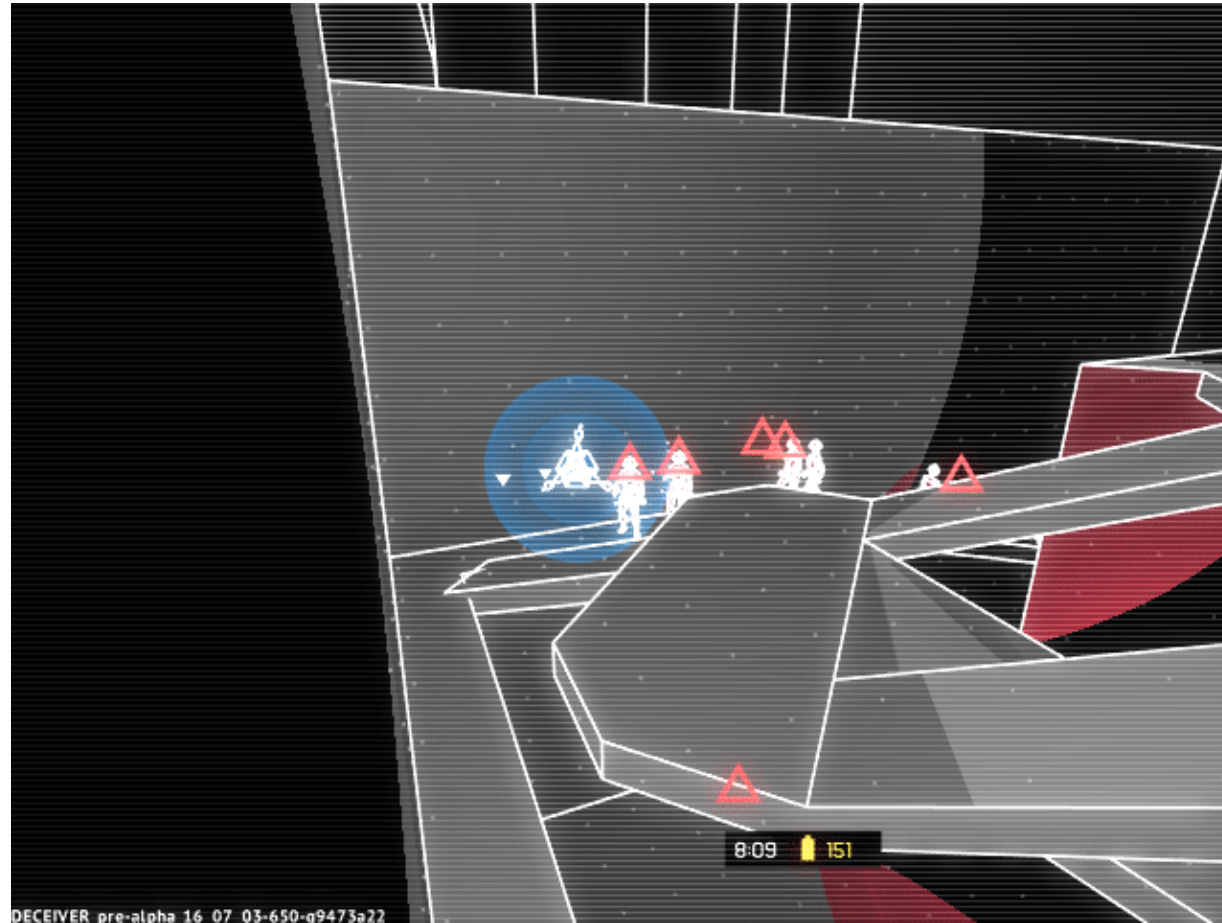
CITM



State, world, objects...



CITM



The invariant of on-line games

USER
GAME
NETW



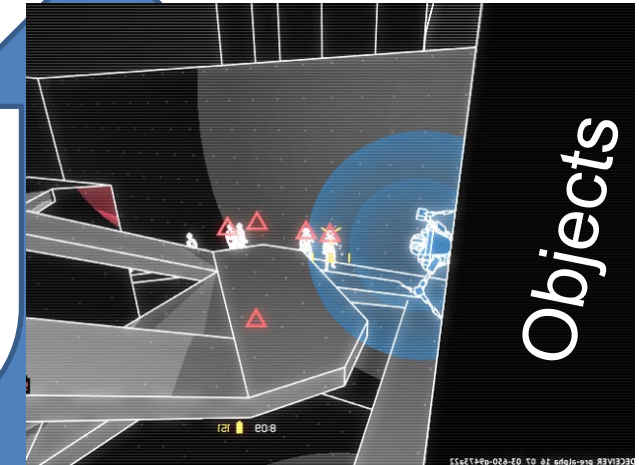
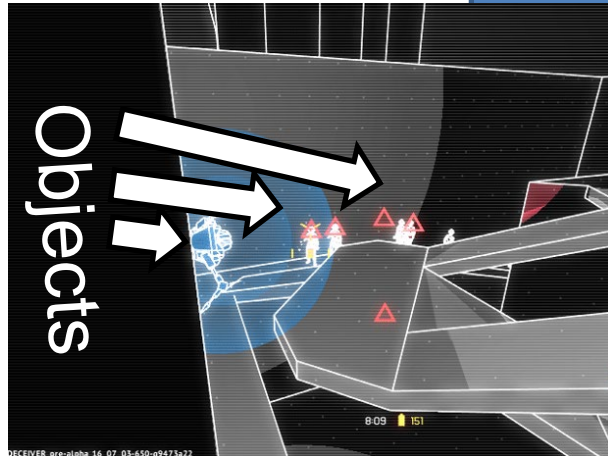
Player 1, click
Player 1 changes state



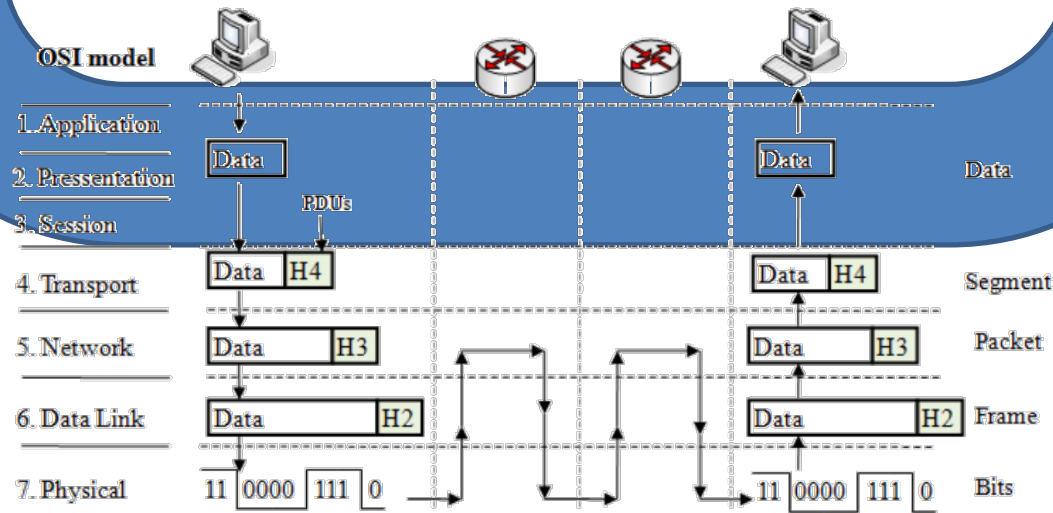
Player 2 receives info
Player 2 changes state

World (state)

World (state)



We want the consequences of actions from player 1 to the state of the simulated world to be seen equally in all players.



The four basics

USER

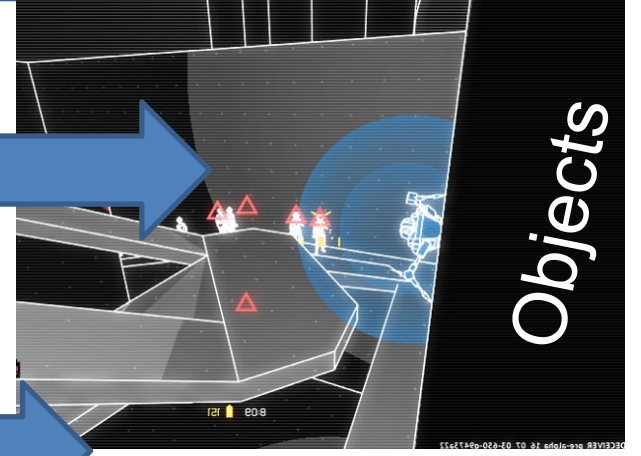
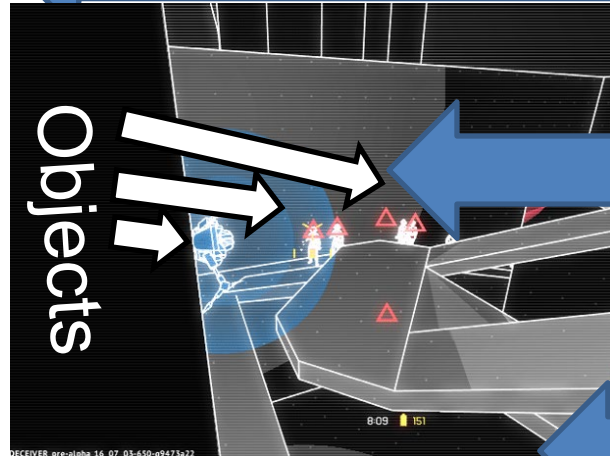


Actions

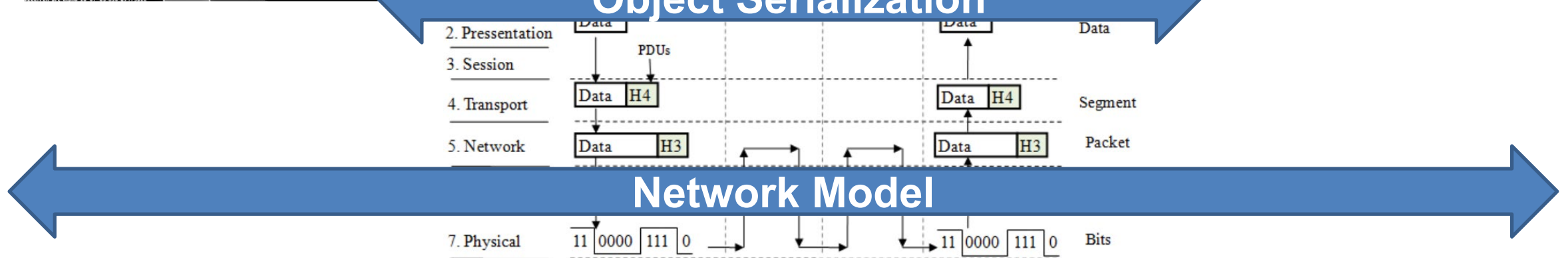
Actions



GAME



NETW



- **Synchronization model:** How is the game state synchronized between different players?
- **Network model:** How are the players connected and who sends game state to whom?
- **Object replication:** Which parts of the game represent the state that is affected by multiple players?
- **Object serialization:** How is the game state serialized so that it can be sent through the network?



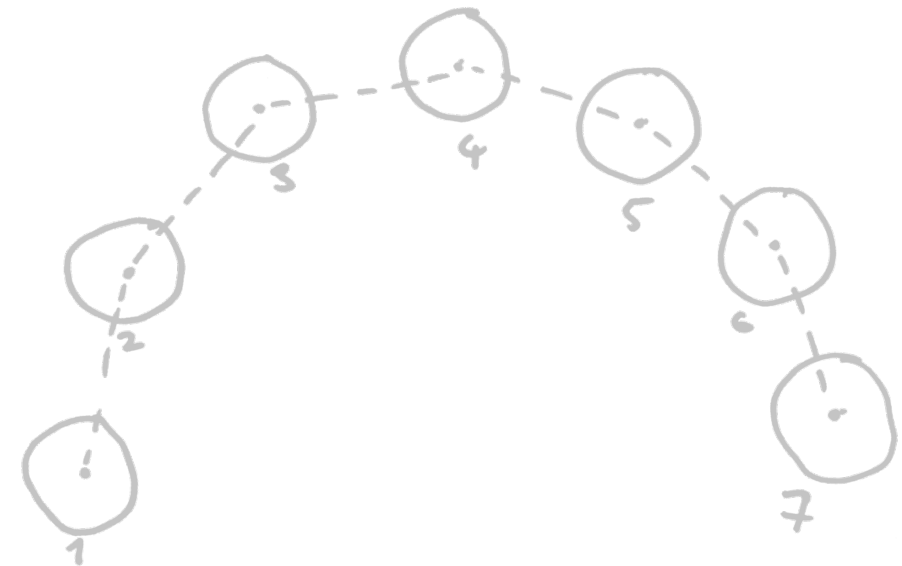
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Synchronization Model

- **Synchronization model:** How is the game state synchronized between different players?
- **Network model:** How are the players connected and who sends game state to whom?
- **Object replication:** Which parts of the game represent the state that is affected by multiple players?
- **Object serialization:** How is the game state serialized so that it can be sent through the network?

1. **Snapshot Interpolation:** the game receives regular snapshots of visual state (e.g. position and rotation) and it interpolates between them
 - Avoids sending one position per frame
 - Still, needs frequent updates for objects that are replicated.
 - The interpolation adds latency, although it is a quite simple operation



Videos (snapshot interpolation)

- <https://youtu.be/UwAw3Mtl1cM> – 60 pps, jitter
- <https://youtu.be/SPm4-KDW-Yk> – 10 pps, no interpolation
- <https://youtu.be/DjQmYrKd92U> – 10 pps, interpolation

Synchronization Model (II)

1. Snapshot Interpolation
2. **State Synchronization:** the clients receive state with extra info (e.g., velocity). There is extrapolation of future state updated/confirmed by further state updates.
 - Need to carefully pick the state to send, as you need more data to better reproduce the state in all clients
 - This makes updates less frequent, which is kind of dangerous.



Videos (state synchronization)



CITM

- <https://youtu.be/egGmccwILdU> - OK
- <https://youtu.be/4ESA5u3bMwY> - Smoothing

1. Snapshot Interpolation
2. State Synchronization
3. **Deterministic Lockstep**: time is divided in “turns”. Inputs are buffered and sent to all players at the end of the epoch. The new state uses the inputs from all players.
 - Inputs are sent instead of objects. This generally leads to less bandwidth use (independent on the number of objects).
 - You have to be very precise on the info to send, so that determinism holds. Synchronized seeds are needed for RNG.
 - Epochs need to be long enough to be relevant. This mostly limits the approach to RTS games (or LAN gaming)

Videos (deterministic lockstep)



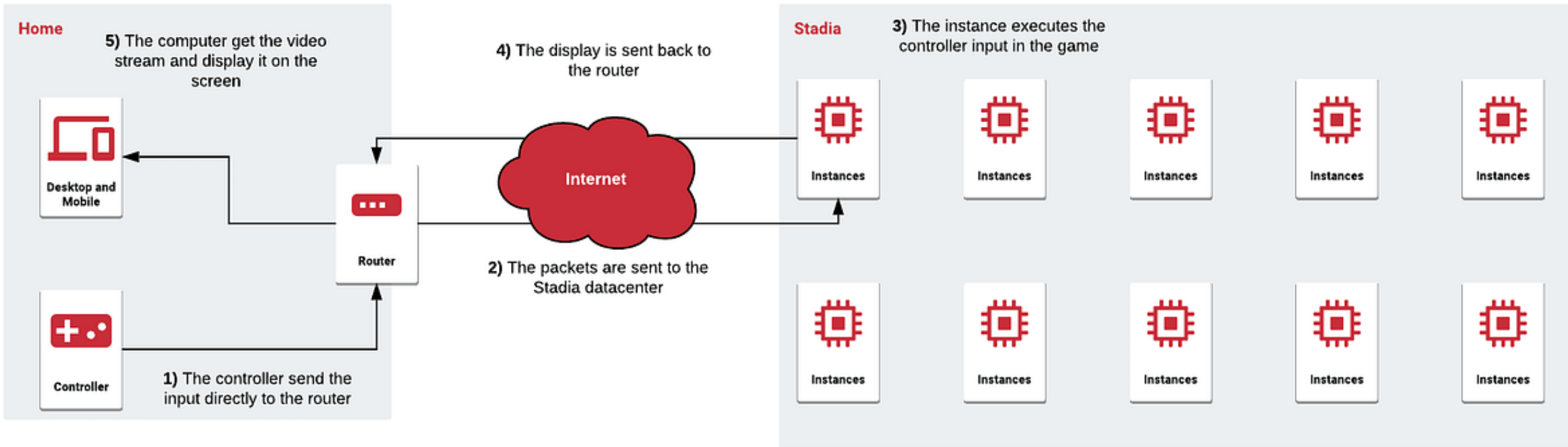
CITM

- <https://youtu.be/wlKEwF2eX6Q> - OK
- <https://youtu.be/x2OP7kuYv7k> - Desync

Synchronization Model (IV)

1. Snapshot Interpolation
2. State Synchronization
3. Deterministic Lockstep
4. **Cloud Gaming:** recent addition by Google, Nvidia, and others. The game logic is run in the cloud!
 - Clients do not have state and hence, by necessity, need to send commands (and not state) to the server.
 - The server, which is now a powerful GPU farm, computes and streams the result of the commands (video/audio).
 - If there is any interpolation or state synchronization happening, it needs to happen at the server.

Synchronization Model (IV)





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

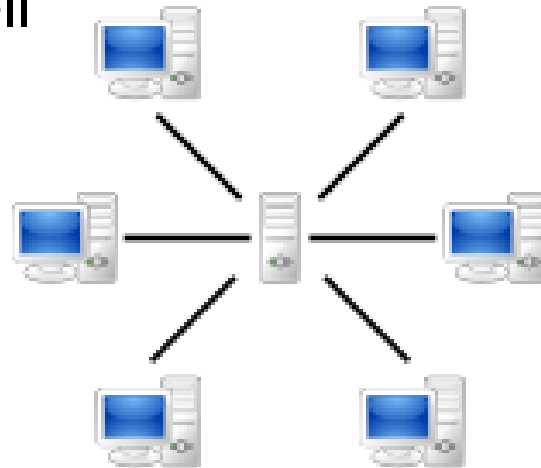
Network Model

- **Synchronization model:** How is the game state synchronized between different players?
- **Network model:** How are the players connected and who sends game state to whom?
- **Object replication:** Which parts of the game represent the state that is affected by multiple players?
- **Object serialization:** How is the game state serialized so that it can be sent through the network?

Remember... Network Architectures

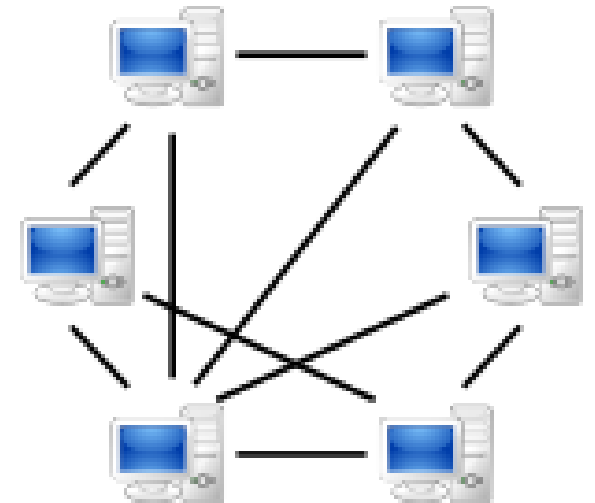
Client-Server Architecture

- Centralized
- The server is in charge of fulfilling requests from clients
- Typical topology: star
- Needs a high-performance and resilient (set of) servers to work
- Does not scale well (from the topology perspective)



Peer-to-Peer (P2P) Architecture

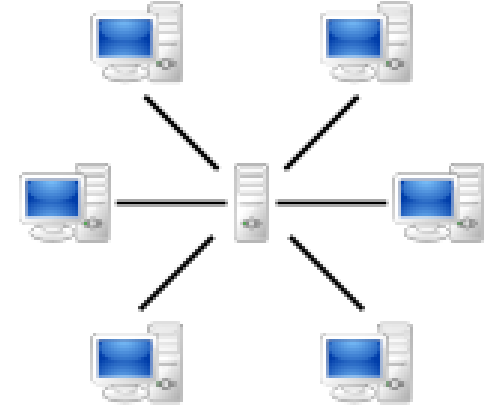
- Distributed
- All nodes are clients and servers, sharing responsibilities
- Typical topology: mesh
- Needs node discovery mechanisms and enhanced security to work
- Scalable (from the topology perspective)



Network Model (I)

1. Client-Server

- Clients send their state changes to the server, who confirms the next state and returns it to the clients (**snapshot interpolation, state synchronization**)
- This is more scalable from the perspective of network use, because you only need $O(2n)$ communications.
- Still less scalable from the perspective of the topology, because:
 - It is asymmetric (server needs more bandwidth)
 - Server generally needs to be dedicated
- Server can be authoritative, meaning that its decisions hold.

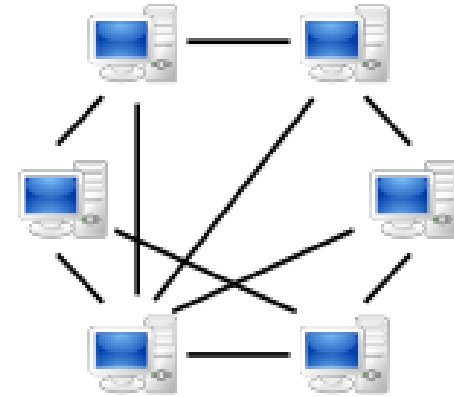


Network Model (II)

1. Client-Server

2. Peer-to-peer

- Clients send their commands to the rest, who decide the next state with **deterministic lockstep**
- This is less scalable from network use perspective, as communications scale as $O(n^2)$
- However, it is more scalable from a topology perspective:
 - Topology is symmetric, no need for strong server
 - Disconnected peers will never disrupt the game.
- Authority is harder to implement.



In summary...

	Snapshot interpolation	State synchronization	Deterministic lockstep	Cloud gaming
<i>Network model</i>	Client-server (usually)	Client-server (usually)	Peer-to-peer (usually)	Client-server (by necessity)
<i>What do I send?</i>	Changes of state caused by my commands	Changes of state; with more info than in snapshot interpolation	My commands	My commands
<i>What do I receive?</i>	Changes of state confirmed by the server	Changes of state confirmed by the server	Commands of other players	Audio/video stream of the game
<i>What do I do with what I receive?</i>	Interpolate between previous state and newly received state.	Correct my state, if wrong, and predict future state	Compute the new state of all objects based on synchronized execution of commands from everyone	Display the stream



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Object Replication

Lab Session 6:

Multiplayer Game

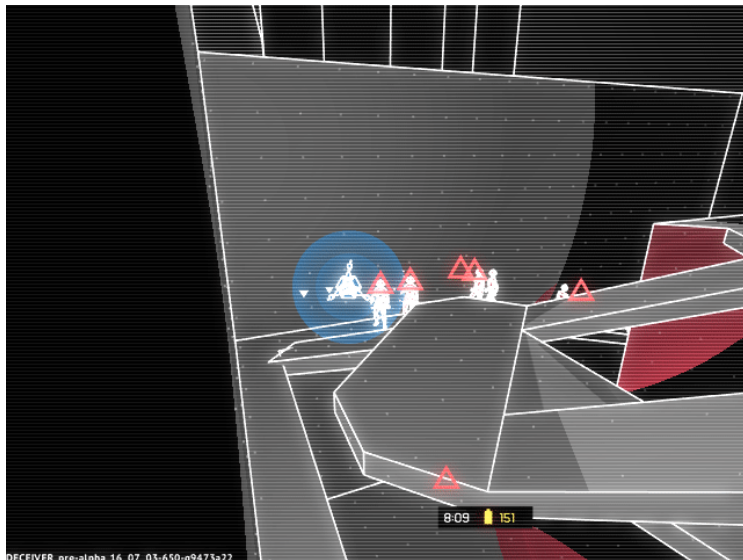
Lab Session 7:

World state replication

- **Synchronization model:** How is the game state synchronized between different players?
- **Network model:** How are the players connected and who sends game state to whom?
- **Object replication:** Which parts of the game represent the state that is affected by multiple players?
- **Object serialization:** How is the game state serialized so that it can be sent through the network?

Object Replication

- Multiplayer games provide a shared experience by keeping a **world state** at each host and exchanging the **state of its objects** to maintain consistency at each host.
- Two types: **active** (for deterministic lockstep) and **passive** (for snapshot interpolation or state synchronization)



What is an object and what is not an object?



Active/Passive Replication

From the lab sessions

Active Replication: Each host receives the update. Each host deterministically updates the game (e.g. Lock-Step)

- Possible issues: Fragility, hard to guarantee that states are all identical, float-point rounding, RND...

Passive Replication: The server receives all inputs from clients. The server broadcasts all updates to each client (e.g. master-slave)

- Possible issues: Server bandwidth/computational power, scalability

Object Replication



- The act of transmitting an object's state from one host to another is called **replication**.
- A good replication involves four steps:
 1. Mark a packet as a packet containing object state
(not all messages will send object state, can be also used to transmit control signals such as: a player has disconnected)



- The act of transmitting an object's state from one host to another is called **replication**.
- A good replication involves four steps:
 1. Mark a packet as a packet containing object state
 2. Uniquely identify the replicated object
(we should have a map of objects associated with a network ID in each computer, so that we can safely identify the same object in all machines only with this ID – avoiding the creation of extra objects, or modification of incorrect ones)

Object Replication

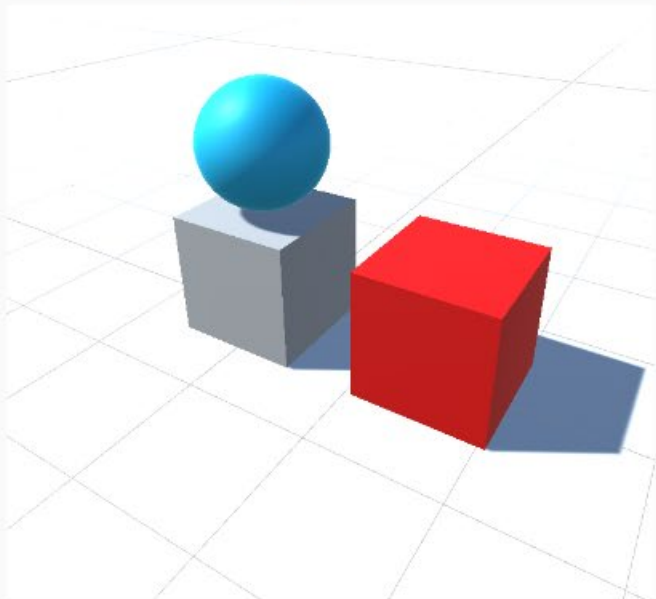


CITM

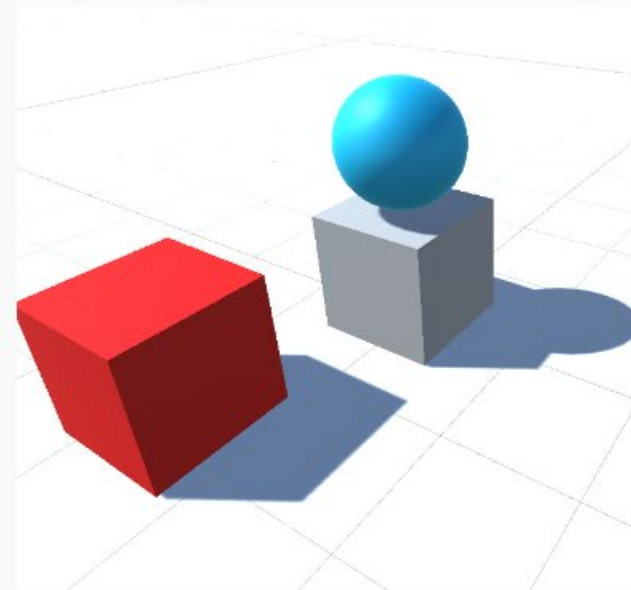
How can the server know which object it has to update?

From the lab sessions

Client



Server



Cube
Cube
Sphere

Network IDs

From the lab sessions

Unique IDs to identify objects over the network

- Every client knows what objects are changing
- Not all objects have a network ID:
 - Have NetID
 - Player
 - Bullets
 - etc.
 - No NetID
 - Static environment elements
 - UI
 - objects with predictable movements (careful!)

- The act of transmitting an object's state from one host to another is called **replication**.
- A good replication involves four steps:
 1. Mark a packet as a packet containing object state
 2. Uniquely identify the replicated object
 3. Indicate the class of the object being replicated
(the first time a client received a new object, it needs to create it. We need to know which kind of object it is. We can try to guess the kind, but we need better ways to do it. An **object creation registry** defines ways to construct an object based on an identifier and the content)

- The act of transmitting an object's state from one host to another is called **replication**.
- A good replication involves four steps:
 1. Mark a packet as a packet containing object state
 2. Uniquely identify the replicated object
 3. Indicate the class of the object being replicated
 4. Serialize the data

Multiple Objects per Packet



CITM

- We want to create packets close to the MTU to amortize the headers, but our objects may be very small.
- So, as we have seen during the course, it may be interesting to send multiple objects in one packet

```
Mark a packet as a packet containing object state
While(size < MTU) {
    Uniquely identify the replicated object
    Indicate the class of the object being replicated
    Serialize the data
}
Send the data
```



World State Replication (naïve)

Transmitter (send modified world)

- Perform the replication for ALL the objects in the world
- Serialize and send as many packets as required, tagged as “replication packets”
- *Apparently in Quake the whole world fit within a single packet*

Receiver (receive modified world)

- Detect a “replication packet” and deserialize the state
- Create new objects that he did not have in the map and appear in the packet
- Update objects that were in the map and in the packet
- Remove objects that did not appear in the packet (meaning that they disappeared)

Partial Changes in World State

- Of course, we cannot afford complete world packets anymore as games are more sophisticated
- We instead represent changes only as **world state deltas** composed by **object state deltas** with an action:
 - Create game object
 - Update game object
 - Destroy game object
- We augment the world update code to support this

"I have object state" header	Action	ObjectID	Data	Action	ObjectID	Data
------------------------------	--------	----------	------	--------	----------	------

Partial Changes in Object State

- To further save space, we can limit the data sent to those fields that change value
- To do this, we can use bit-fields to represent those properties that we send and expect to change
- Therefore, in each world update we only modify those objects that changed, and only in the attributes that changed

"I have object state" header	Action	ObjectID	Changed Fields	Data for Field i	Data for Field j	...	Data for Field k
---------------------------------	--------	----------	-------------------	---------------------	---------------------	-----	---------------------

Action	ObjectID	Changed Fields	Data for Field i	Data for Field j	...	Data for Field k
--------	----------	-------------------	---------------------	---------------------	-----	---------------------

Linking Objects

From the lab sessions

Register ID: Create a new ID (server side) vs. use a specific ID (client side)

Find objects by ID (both in network and application sides)

Unregister ID: Remove object from network IDs

- Create game object
- Update game object
- Destroy game object

- https://youtu.be/pY2xkWkJ_5U - Complete world is sent at every frame with as much compression as possible.
- https://youtu.be/YqjDgtj_xY0 - Same, with world state delta and other related optimizations.



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Object Serialization

- **Synchronization model:** How is the game state synchronized between different players?
- **Network model:** How are the players connected and who sends game state to whom?
- **Object replication:** Which parts of the game represent the state that is affected by multiple players?
- **Object serialization:** How is the game state serialized so that it can be sent through the network?

Multiple Objects per Packet

- We want to create packets close to the MTU, but our objects may be very small.
- As we have seen in the example of Starsiege: Tribes, it may be interesting to send multiple objects in one packet

Mark a packet as a packet containing object state

```
While(size < MTU) {
```

```
    Uniquely identify the replicated object
```

```
    Indicate the class of the object being replicated
```

```
    Serialize the data
```

```
}
```

```
Send the data
```

Serialized object data



- See slides in lesson 2.1 of the course to remember about:
 - Having Read/Write methods in all your classes
 - Endianness
 - Linking
 - Embedding



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Relevant examples

Example: Age of Empires (I)

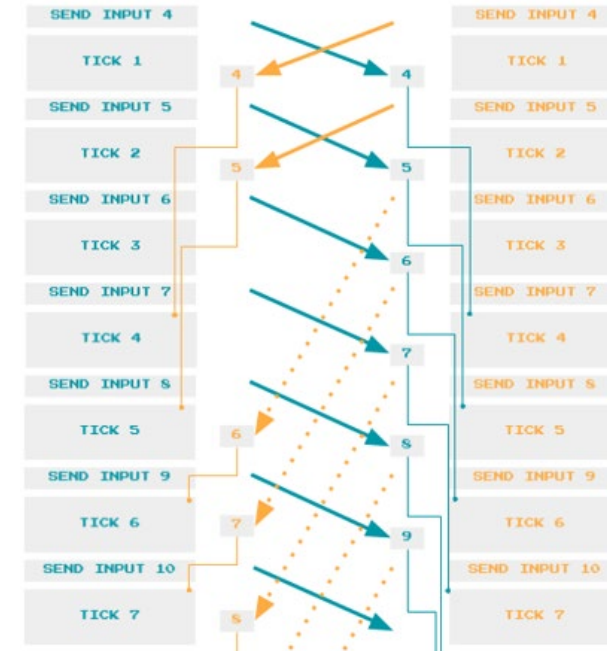
- First on-line multiplayer version released in 1990s
- Dial-up access allowed (28-56 Kbps) – latency: 100s ms
- With such slow speeds, the game would support 8 player battles with 50 units per player.
- How? **With deterministic lockstep and peer-to-peer architecture**
- Instead of sending game state (position and actions of each unit), we send the valid commands of each user.



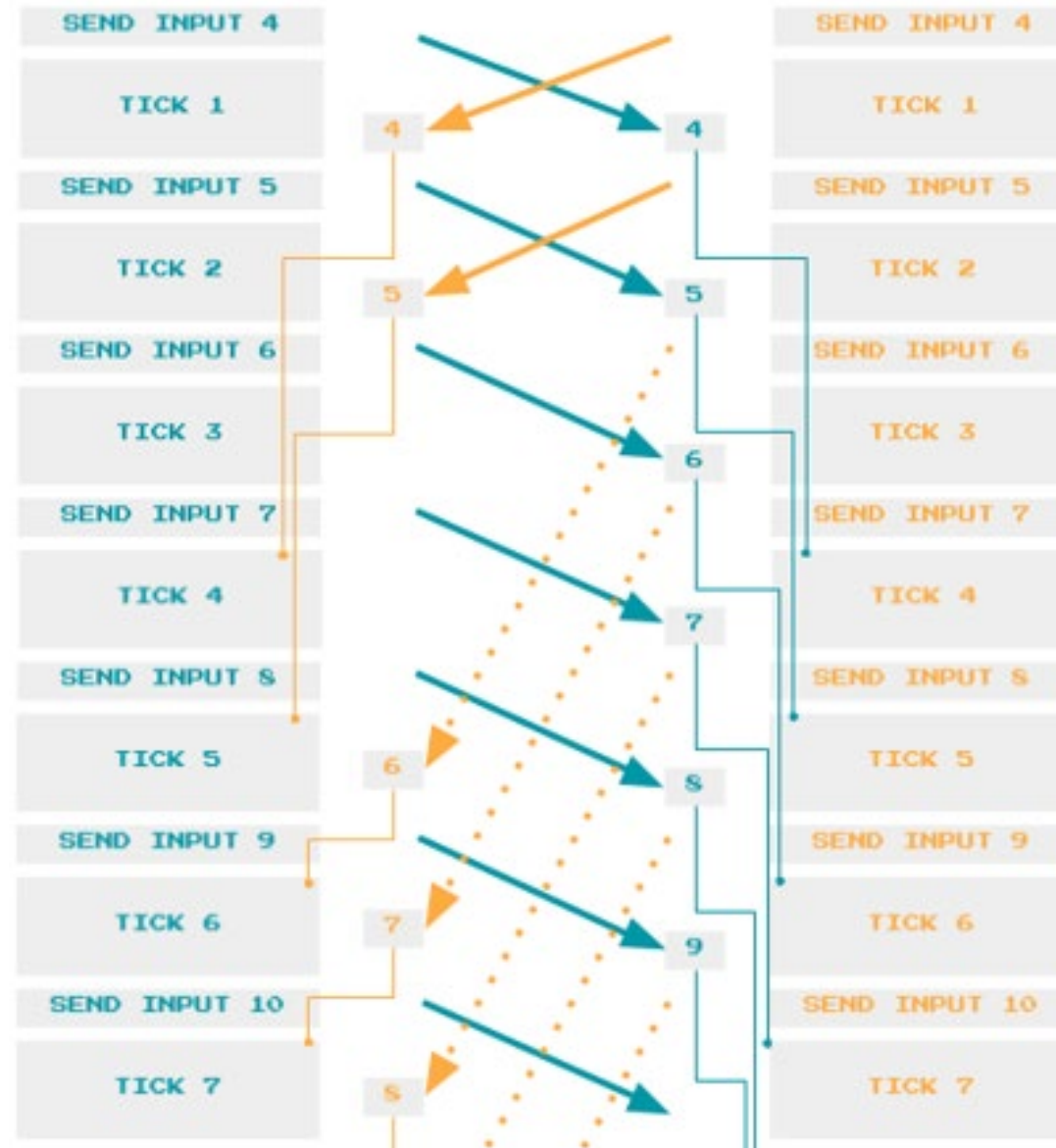
Example: Age of Empires (II)



- Challenge? Turn timers.
 - AoE used 200ms by default. All commands were buffered for this amount of time and sent.
 - AoE used a “turn delay”. When we send inputs for turn 4, we are executing turn 1. That gives a slack of 600ms for players who may be lagging.
 - In the end, the game would go as fast as the slowest connected player, because all players need all inputs in lockstep.
 - AoE implemented a series of pings to the rest of peers in the P2P architecture to calculate delays. With this, new timers were calculated to adjust the rate to varying connections.



Example: Age of Empires (II)

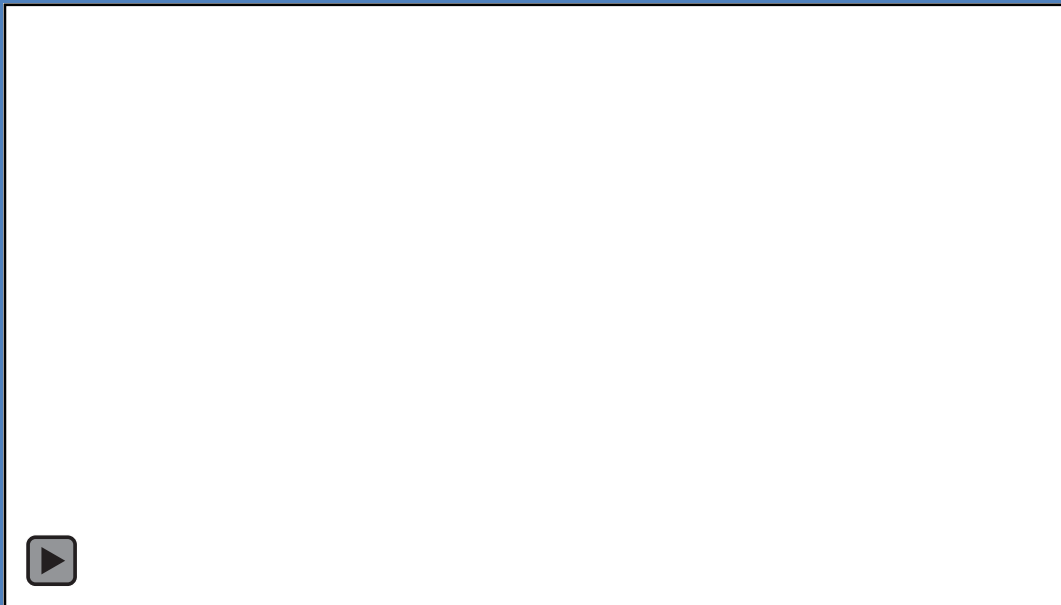


Example: Age of Empires (II)



CITM

This is also called delay-based deterministic lockstep and it's used in fighting games



<https://words.infil.net/w02-netcode-p3.html>

Example: Age of Empires (III)

- Challenge? Synchronization.
 - AoE may not have it, but RTS games often needs random checks to assess damage, for instance.
 - All players should have the same pseudo-random seed with the same pseudo-random number generator.
 - It is crucially important that **all machines have the same results, with high precision**. Otherwise, the “butterfly effect” occurs.
 - Due to these same reasons, it is not hard to detect players that cheat and make the game state to become inconsistent. This was a plus, security-wise.

Example: Age of Empires (IV)

- Although all inputs are important, AoE used UDP for fast delivery with custom reliability mechanisms.
 - Command ordering, drop detection, and resending performed at the application layer.
- A simple but significant optimization data serialization was to **filter unnecessary commands** (furious clicks in battle)
- Due to the deterministic lockstep, all players can record all comments and have nice record-and-play capabilities
 - Think that back in the time, recording a video or streaming was downright crazy.

Example: Starsiege Tribes (I)

- First on-line multiplayer version released in 1998
- Dial-up access allowed (28-56 Kbps) – latency: 100s ms
- With such slow speeds, the game supported 128 players.
- How? **With state synchronization in a client-server architecture**, where the client would probably be in a University or Internet Cafe.
- Game state was sent, not commands, through different mechanisms to ensure a consistently good experience



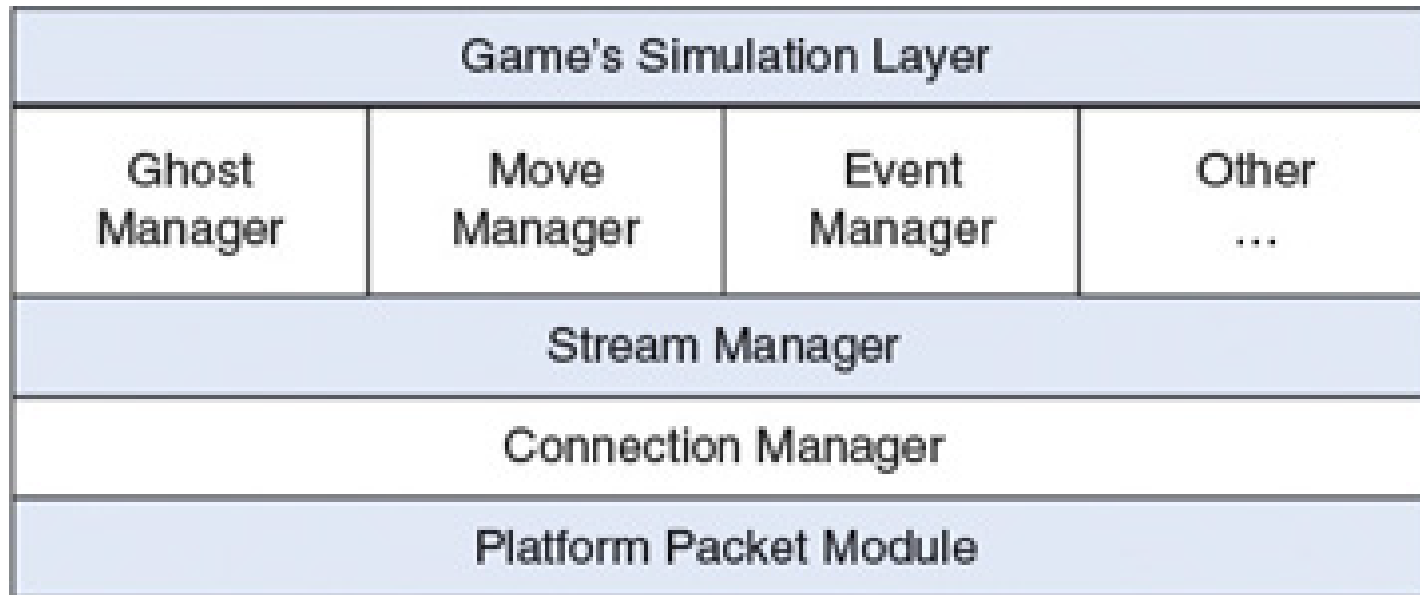
Example: Starsiege Tribes (II)



- Data categories:
 - **Non-guaranteed data:** it is not essential. Can be dropped if bandwidth needs to be saved.
 - **Guaranteed data:** critical data that needs to arrive and be ordered, like someone shooting.
 - **“Most recent state” data:** this is data that is constantly updated, so we only need the most recent version (if middle updates are dropped, could be fine). E.g: player MP.
 - **Guaranteed quickest data:** this is critical data that, moreover, needs to arrive as quick as possible. Movement information is of this type.

Example: Starsiege Tribes (III)

- All communications use UDP, but Tribes implements a layered model to implement custom reliability and support the different data types
 - Shows how layers can appear at application layer and above

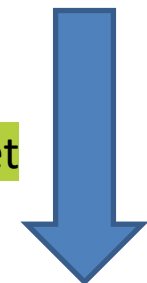


The game code being executed at FPS

Runtime managers for replication of different aspects

Layers managing replication priorities, connections, custom reliability

Wrapper for UDP socket



UDP
IP
Ethernet...

Example: Starsiege Tribes (III)

- All communications use UDP, but Tribes implements a layered model to implement custom reliability and support the different data types

Bungie's Networking Stack

Layer	Purpose
Game	Runs the game
Game Interface	Extract and apply replicated data
Prioritization	Rate the priority of all possible replication options
Replication	Protocols with various reliability guarantees
Channel Manager	Flow and congestion control
Transport	Send & receive on sockets

BUNGI

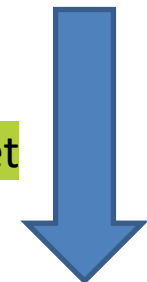
Application layer and above

The game code being executed at FPS

Runtime managers for
replication of different aspects

Layers managing replication
priorities, connections,
custom reliability

Wrapper for UDP socket



UDP
IP
Ethernet...

Example: Starsiege Tribes (IV)



- **Event Manager:** queue of events generated by the simulation, that need to be validated by the server and confirmed. Example: “player fired”.
- **Move Manager:** this attempts to manage how the player moves as it crucial for the quality of experience.
- **Ghost Manager:** replicates “ghost” dynamic object that are considered relevant by the game’s simulation layer (assigning a given ID, state, priority). It is important to guarantee the most recent state for these objects ordered by priority.

Example: Starsiege Tribes (IV)



- **Stream manager:** this layer decides what data to actually send and with which priority (decided by the runtime managers above). This depends on the maximum speed that clients allow, by setting the send interval and the size of the messages.
- **Connection manager:** provides delivery status notifications and see who is connected when
- **Platform Packet Module:** this basically provides a wrapping of the UDP sockets to serialize and then parse the messages

- Extra reads (Atenea)
 - 1500 Archers on a 28.8: Network Programming in Age of Empires and Beyond.
 - Gaffer on Games: Deterministic Lockstep
 - Gaffer on Games: Snapshots and Interpolation



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Impact on programming

3.1 Summary and General Aspects

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs